

# Descripción del problema y objetivos

El problema radica en la **falta de una herramienta personalizada y centralizada** que permita no solo registrar el consumo de animés, sino también la clasificación eficiente mediante criterios específicos (género, estado, calificación) y organización en listas temáticas personalizadas.

Además, desde una perspectiva de ingeniería de software, el desafío consiste en modelar este dominio (que incluye diferentes tipos de entidades como Series y Películas con atributos distintivos) dentro de un sistema robusto, escalable y mantenible, evitando el diseño monolítico y el código fuertemente acoplado.

## Objetivo General

Diseñar y desarrollar una **aplicación de escritorio en Java (Swing)** que permita la gestión integral, clasificación y recomendación de un catálogo personal de animé, aplicando los conceptos del Paradigma Orientado a Objetos.

## Objetivos Específicos

1. **Arquitectura de Software:** Implementar una separación clara de responsabilidades mediante una arquitectura en capas (**Modelo, Vista, Controlador/Servicio, Repositorio**), asegurando un bajo acoplamiento y una alta cohesión.
2. **Diseño Extensible (SOLID y GRASP):**
  - a. Aplicar principios **SOLID**, especialmente el *Open/Closed Principle (OCP)* para permitir la incorporación de nuevos criterios de búsqueda y ordenamiento sin modificar el código base.
  - b. Utilizar patrones **GRASP** (como *Controller, Creator* e *Indirection*) para asignar correctamente las responsabilidades a las clases.
3. **Modelado del Dominio:** Representar correctamente la relación entre entidades mediante el uso de **herencia y polimorfismo** (diferenciando **AnimeSerie** de **AnimePelícula** bajo el Principio de Sustitución de Liskov).
4. **Robustez y Persistencia:** Garantizar la integridad de los datos mediante validaciones de dominio, manejo de excepciones personalizadas y persistencia de la información en archivos, evitando cierres abruptos de la aplicación.
5. **Experiencia de Usuario (UX):** Proveer una interfaz gráfica intuitiva que permita realizar operaciones CRUD, filtrados dinámicos y gestión de listas personalizadas de manera eficiente.

# Casos de Uso

A continuación se describen los casos de uso principales que cubren la funcionalidad crítica del sistema.

## CU-01: Registrar Nuevo Animé

**Actor:** Usuario. **Descripción:** Permite al usuario ingresar una nueva serie o película al catálogo general.

**Precondición:** El título del animé no debe existir previamente en el sistema. **Flujo Principal:**

1. El usuario selecciona la opción "Nuevo Animé".
2. El sistema solicita seleccionar el tipo de registro: "Serie" o "Película".
3. El usuario ingresa los datos obligatorios: Título, Año, Estudio y el dato específico según el tipo (Cantidad de Capítulos para series o Duración en minutos para películas).
4. El sistema valida que el título no esté vacío y que los valores numéricos sean positivos.
5. El sistema verifica que no exista un duplicado mediante el título.
6. El sistema crea el objeto correspondiente y lo persiste en el archivo de almacenamiento.
7. La tabla principal se actualiza mostrando el nuevo registro.

## CU-02: Gestionar Listas Personalizadas

**Actor:** Usuario. **Descripción:** Permite la creación de listas temáticas (ej. "Favoritos") y la adición de animes existentes a dichas listas. **Precondición:** Deben existir animes registrados en el catálogo. **Flujo Principal:**

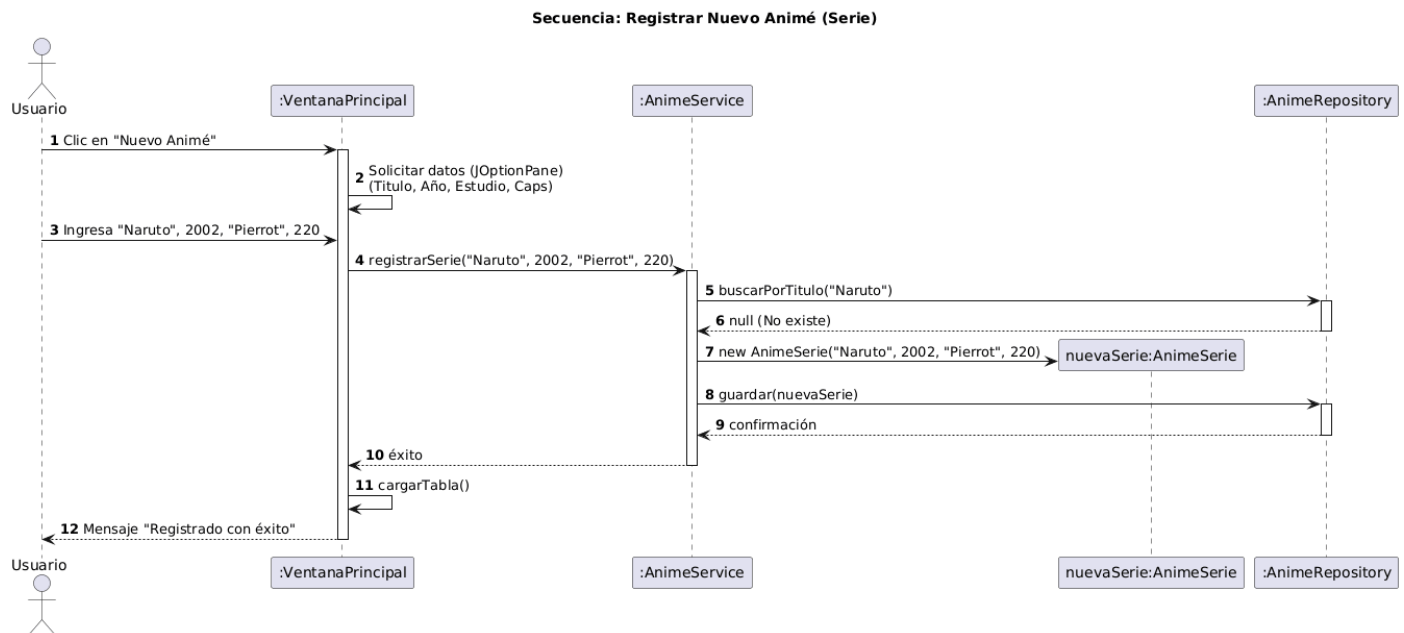
1. **Creación:** El usuario selecciona "Nueva Lista" e ingresa un nombre único. El sistema valida que no exista otra lista con el mismo nombre y la crea vacía.
2. **Asignación:** El usuario selecciona un animé de la tabla principal y elige la opción "Agregar a Lista...".
3. El sistema despliega las listas disponibles.
4. El usuario selecciona la lista de destino.
5. El sistema registra la relación (referencia) entre la lista y el animé, guardando los cambios en el archivo de listas (`listas.csv`).

## CU-03: Filtrar Catálogo (Búsqueda)

**Actor:** Usuario. **Descripción:** Permite al usuario visualizar un subconjunto de animes que cumplan con un criterio específico (Estrategia). **Flujo Principal:**

1. El usuario despliega el menú de filtros y selecciona un criterio (Título, Género, Estado, Rango de Años o Calificación).
2. El sistema solicita el valor de búsqueda correspondiente (ej. un texto parcial para título o un rango numérico para años).
3. El usuario ingresa el valor y confirma.
4. El sistema aplica la estrategia de filtrado seleccionada sobre la colección completa.
5. La interfaz gráfica se actualiza mostrando únicamente los animes que cumplen con el criterio.

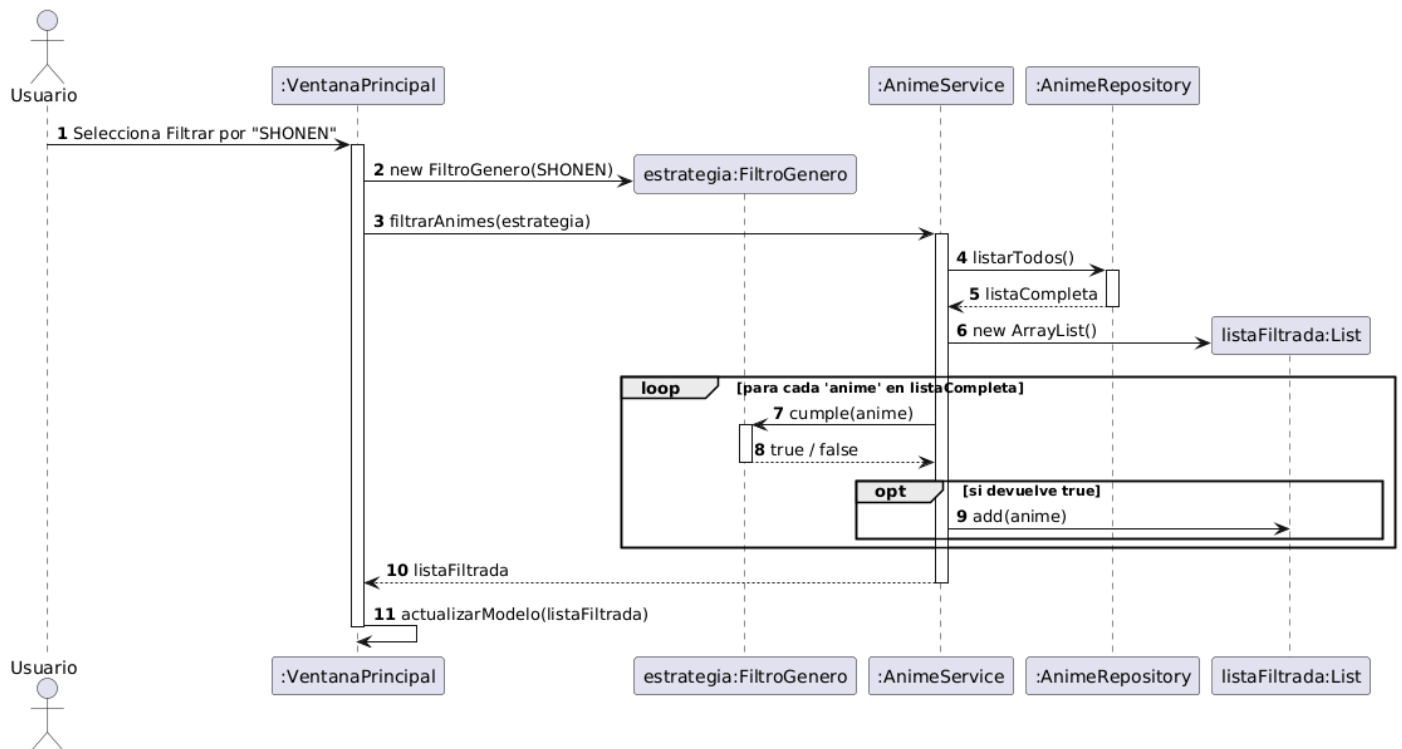
# Diagramas de secuencia



## 1. Descripción del Caso de Uso: "Registrar Nuevo Animé"

- Inicio del Evento:** El usuario interactúa con la `VentanaPrincipal` (UI) ingresando los datos (Título, Año, Estudio, etc.) y confirmando la acción.
- Delegación (Controller):** La UI invoca al método `registrarSerie(...)` del `AnimeService`, pasándole solo los datos primitivos. Esto mantiene a la UI desacoplada de la lógica de negocio.
- Validación:** El `AnimeService` primero colabora con el `AnimeRepository` llamando a `buscarPorTitulo()`. Si el animé ya existe, se corta el flujo lanzando una excepción (validación de regla de negocio).
- Creación (Creator):** Una vez validados los datos, el `AnimeService` instancia el nuevo objeto: `new AnimeSerie(...)`. El servicio es el "Creador" porque posee los datos necesarios para inicializarlo.
- Persistencia (DIP):** Finalmente, el Servicio solicita al `AnimeRepository` que guarde la nueva instancia mediante el mensaje `guardar()`. El servicio desconoce si se guarda en un archivo o base de datos, cumpliendo con la Inversión de Dependencias.
- Respuesta:** El control regresa a la UI, que actualiza la tabla visual para mostrar el nuevo registro al usuario.

### Secuencia: Filtrar Animes (Patrón Strategy)



## 2. Descripción del Caso de Uso: "Aplicar Filtro (Strategy)"

- Configuración de la Estrategia:** Cuando el usuario selecciona un filtro (ej. "Por Género: SHONEN"), la `VentanaPrincipal` instancia la estrategia concreta correspondiente: `new FiltroGenero(SHONEN)`.
- Invocación Polimórfica:** La UI llama al método `filtrarAnimes(estrategia)` del `AnimeService`. El Servicio recibe la interfaz `CriterioBusqueda`, sin saber qué implementación específica está llegando.
- Obtención de Datos:** El Servicio solicita la lista completa de animes al `AnimeRepository` mediante `listarTodos()`.
- Iteración y Delegación:** El Servicio recorre la lista de animes. Para cada uno, delega la decisión de inclusión a la estrategia: `estrategia.cumple(anime)`.
- Si la estrategia es de Género, verificará el `Set` de géneros. El Servicio permanece "cerrado" a modificaciones (OCP) porque esta lógica es externa a él.
- Construcción de Resultados:** Si el método `cumple()` retorna `true`, el animé se agrega a una lista temporal.
- Retorno:** Finalmente, el Servicio devuelve la lista filtrada a la UI, que se encarga de redibujar la tabla (`actualizarModelo`).

# Descripción del diseño

## Descripción del Diagrama de Clases

El diseño del sistema se estructura en cuatro paquetes principales (`model`, `repository`, `service`, `ui`), respetando una arquitectura en capas para asegurar el bajo acoplamiento y la alta cohesión.

### 1. Paquete Model (Dominio)

Representa las entidades principales y las reglas de negocio básicas.

- **Clase Abstracta `Anime`:** Es la entidad base del sistema. Define los atributos comunes (`titulo`, `anioLanzamiento`, `estudio`, `calificacion`, `estado`) y gestiona los géneros mediante un `Set<GeneroAnime>`. Define el método abstracto `getDuracionTotal()`, obligando a las subclases a implementar su propia lógica de cálculo.
- **Clases Concretas `AnimeSerie` y `AnimePelícula`:** Heredan de `Anime`.
  - `AnimeSerie`: Añade el atributo `cantidadCapitulos` e implementa el cálculo de duración basado en capítulos.
  - `AnimePelícula`: Añade el atributo `duracion` (en minutos).
  - Esta estructura aplica herencia y polimorfismo, cumpliendo con el principio **LSP (Liskov)**.
- **Enums:** `EstadoAnime` y `GeneroAnime` tipifican los estados y categorías posibles, evitando el uso de cadenas de texto arbitrarias.
- **`ListaPersonalizada`:** Representa las listas creadas por el usuario. Mantiene una relación de **agregación** con `Anime` (cardinalidad `0..*`), indicando que una lista contiene referencias a animes, pero estos pueden existir independientemente de la lista.

### 2. Paquete Repository (Persistencia)

Encargado del almacenamiento y recuperación de datos.

- **Interfaz `AnimeRepository`:** Define el contrato para las operaciones CRUD (guardar, eliminar, buscar, listar). Su uso es fundamental para cumplir con el principio **DIP (Inversión de Dependencias)**.
- **`RepositoryAnimeArchivo`:** Implementa la interfaz `AnimeRepository`. Se encarga de la persistencia física en archivos CSV, parseando las líneas de texto a objetos y viceversa.
- **`RepositoryListas`:** Gestiona la persistencia específica de las listas personalizadas y la reconstrucción de las relaciones entre listas y animes existentes.

### 3. Paquete Service (Lógica de Negocio)

Actúa como intermediario y orquestador del sistema.

- **`AnimeService`:** Es la clase central (Patrón **Controller** de GRASP). Recibe las peticiones de la UI, valida las reglas de negocio, gestiona la creación de objetos (Patrón **Creator**) y delega la persistencia a los repositorios. Depende de la abstracción `AnimeRepository` y no de su implementación concreta.
- **Patrón Strategy (Búsqueda):** Se define la interfaz `CriterioBusqueda` con el método `cumple(Anime)`. Existen múltiples implementaciones concretas (`FiltroGenero`, `FiltroTitulo`, `FiltroAnio`, `FiltroEstado`, `FiltroCalificacion`). Esto permite extender los filtros sin modificar el servicio, cumpliendo con el principio **OCP (Open/Closed)**.

## 4. Paquete UI (Interfaz Gráfica)

Contiene los componentes visuales basados en Swing.

- **VentanaPrincipal:** Es el punto de entrada de la aplicación. Mantiene una referencia al `AnimeService` para delegar todas las acciones del usuario.
- **DialogoEditar:** Ventana modal encargada de la modificación de datos de un animé específico.

## 5. Manejo de Excepciones

Se implementa una jerarquía de excepciones propias para el manejo robusto de errores:

- `AnimeException` como clase base.
- `ValidacionException` para errores de lógica de negocio (entradas inválidas).
- `PersistenciaException` para errores de entrada/salida.

# Aplicación de Patrones GRASP y Principios SOLID

El diseño del sistema se fundamentó en la asignación correcta de responsabilidades y la creación de una arquitectura robusta y flexible. A continuación, se detalla la aplicación de cada patrón y principio requerido.

## Patrones GRASP

### 1. Controller (Controlador)

- **Aplicación:** La clase `AnimeService` actúa como el controlador del sistema.
- **Justificación:** Es el primer objeto más allá de la capa de interfaz de usuario (`VentanaPrincipal`) que recibe y coordina las operaciones del sistema. Desacopla la vista de la lógica de negocio y del acceso a datos.

### 2. Creator (Creador)

- **Aplicación:** `AnimeService` es responsable de instanciar las clases `AnimeSerie` y `AnimePelícula`.
- **Justificación:** El servicio recibe los datos primitivos (título, año, etc.) desde la UI y posee la información necesaria para inicializar correctamente los objetos del dominio antes de enviarlos a guardar.

### 3. Information Expert (Experto en Información)

- **Aplicación:** El método `getDuracionTotal()` en las clases `AnimeSerie` y `AnimePelícula`.
- **Justificación:** Cada subclase es la experta en calcular su propia duración (una multiplicando capítulos, la otra devolviendo minutos directos) porque son las que poseen los atributos privados (`cantidadCapitulos`, `duracionMinutos`) necesarios para ese cálculo.

#### 4. Low Coupling (Bajo Acoplamiento)

- **Aplicación:** La separación estricta en capas (`ui` -> `service` -> `repository`).
- **Justificación:** La `VentanaPrincipal` no conoce la existencia de archivos CSV, ni el `AnimeService` sabe cómo se dibuja la tabla. Esto permite realizar cambios en una capa (ej. cambiar el diseño de la ventana) sin afectar a las demás.

#### 5. High Cohesion (Alta Cohesión)

- **Aplicación:**
  - `RepositorioAnimeArchivo`: Se encarga **únicamente** de la persistencia (lectura/escritura de archivos).
  - `AnimeService`: Se encarga **únicamente** de la lógica de negocio y validaciones.
- **Justificación:** Cada clase tiene un propósito claro y enfocado, facilitando su comprensión y mantenimiento.

#### 6. Polymorphism (Polimorfismo)

- **Aplicación:**
  - El manejo de la lista `List<Anime>` que contiene tanto series como películas.
  - La interfaz `CriterioBusqueda`, donde el método `cumple(Anime)` se comporta distinto según si la instancia es un `FiltroGenero` o un `FiltroAnio`.
- **Justificación:** Permite tratar objetos diferentes de manera uniforme y eliminar sentencias `if-else` complejas en la lógica de filtrado.

#### 7. Pure Fabrication (Fabricación Pura)

- **Aplicación:** La clase `RepositorioAnimeArchivo`.
- **Justificación:** No representa un concepto del dominio real (no es un "Animé" ni una "Lista"), sino una clase técnica creada artificialmente para encargarse de la responsabilidad de guardar datos en el disco duro, manteniendo al modelo de dominio limpio de lógica de infraestructura.

#### 8. Indirection (Indirección)

- **Aplicación:** La interfaz `AnimeRepository` actúa como intermediario entre el Servicio y la implementación real del archivo.
- **Justificación:** Crea un punto de conexión flexible que evita que el servicio dependa directamente de la clase que maneja archivos.

#### 9. Protected Variations (Variaciones Protegidas)

- **Aplicación:** El uso de interfaces (`AnimeRepository`, `CriterioBusqueda`) para envolver los puntos de inestabilidad o cambio.
- **Justificación:** Si decidimos cambiar el sistema de archivos por una base de datos SQL en el futuro, o agregar nuevos filtros de búsqueda, el resto del sistema está protegido y no sufrirá cambios.

## Principios SOLID

## 1. SRP - Single Responsibility Principle (Principio de Responsabilidad Única)

- **Aplicación:** Se separó la lógica de validación (`AnimeService`) de la lógica de persistencia (`RepositorioAnimeArchivo`).
- **Justificación:** Si cambia la regla de negocio sobre cómo validar un título, solo modificamos el Servicio. Si cambia el formato del archivo CSV, solo modificamos el Repositorio. Cada clase tiene una única razón para cambiar.

## 2. OCP - Open/Closed Principle (Principio Abierto/Cerrado)

- **Aplicación:** El sistema de filtrado mediante la interfaz `CriterioBusqueda`.
- **Justificación:** El sistema está **abierto a la extensión** (podemos crear nuevas clases como `FiltroEstudio` o `FiltroDirector`) pero **cerrado a la modificación** (no necesitamos tocar el código del método `filtrarAnimes` en el Servicio para que los nuevos filtros funcionen).

## 3. LSP - Liskov Substitution Principle (Principio de Sustitución de Liskov)

- **Aplicación:** La herencia entre `Anime` (clase base) y sus hijos `AnimeSerie` y `AnimePelicula`.
- **Justificación:** Cualquier funcionalidad del sistema que espere un objeto de tipo `Anime` (como las `ListaPersonalizada` o los métodos de ordenamiento) funciona correctamente recibiendo una instancia de `AnimeSerie` o `AnimePelicula` indistintamente, sin romper el programa.

## 4. ISP - Interface Segregation Principle (Principio de Segregación de Interfaz)

- **Aplicación:** La interfaz `CriterioBusqueda` es pequeña y específica (sólo tiene un método `cumple`).
- **Justificación:** No obligamos a las clases de filtro a implementar métodos que no necesitan. Por ejemplo, un filtro no necesita métodos de ordenamiento ni de guardado, solo de validación.

## 5. DIP - Dependency Inversion Principle (Principio de Inversión de Dependencias)

- **Aplicación:** `AnimeService` depende de la abstracción `AnimeRepository` (interfaz), no de la clase concreta `RepositorioAnimeArchivo`.
- **Justificación:** Los módulos de alto nivel (lógica de negocio) no dependen de los detalles de bajo nivel (archivos). Ambos dependen de abstracciones. Esto se materializa inyectando el repositorio en el constructor del servicio.